

④ AI 활용 기술 문서 — 「수화기 너머의 벽」

한 줄: 이 문서는 "AI를 썼다"는 자랑이 아니라, **AI를 어디까지 믿고 어디서 끊을지를 설계·지휘한 기록**이다.

게임	수화기 너머의 벽 (병원 경영 · 응급 수용 시뮬레이션)
대회	NAN 2026 — NHN Game × AI 해커톤 · 사전 과제
제출자	개인 참여(솔로)
플레이(배포)	https://goospel.github.io/hospital-sim/ (설치·API 키 없이 브라우저에서 바로)
소스(공개)	https://github.com/Goospel/hospital-sim — 커밋 기록 유지(결정론·TDD 이력 그대로)
문서 상태	작업 초안 (2026-07-22 갱신 — §6 실제 지시 프롬프트 로그 채움) · 실LLM 극소판(극소판)·풀 2콜(x8) 착수 후 런타임 로그·스크린샷을 채워 P7(8/3~8/7)에 최종화
시각화·PDF판	🔗 디렉터의 운영체제 — HTML→PDF · 라이브 아티팩트로 성장 중

🔴 표기 규칙 — 아직 존재하지 않는 산출물(실제 프롬프트↔응답 로그, 스크린샷, 라이브 지연 실측치)은 본문에 ⏳ [자리표시자 - ...] 로 남겨, 최종 집필 때 무엇을 채울지 바로 보이게 했다. 초안 시점 게임의 실제 런타임은 100% 결정론 풀백이며(무기로 완주 가능), 라이브 LLM은 그 위에 비파괴로 얹는 가산 레이어다.

0. 이 문서의 입장 — "쓴 사람"이 아니라 "끊는 자리를 정한 사람"

NAN 2026은 이렇게 묻는다.

"AI를 활용하는 사람은 많습니다. 우리가 찾는 사람은 **AI의 다음 단계를 설계할 디렉터**입니다."

그래서 이 문서는 두 개의 축으로 AI를 증명한다. 그리고 둘의 위계를 분명히 한다.

- 주(主) — 디렉팅(B축): "**AI를 어떻게 지휘해 이 게임을 만들었나.**" 무엇을 AI에 맡기고, 어디서 그 권한을 끊고, AI가 낸 안을 어디서 사람이 검증·기각했는가. → §2, §3, §6(실제 지시 프롬프트 원문)
- 종(從) — 게임 속 AI(A축): "**게임 안에서 AI를 어떻게 부리나.**" 받는 콜 담당자 대사, 결말 팩트 병치. → §2(대사콜), §4

핵심 주장은 하나다. 이 게임에서 가장 인상적인 "**게임 속 AI**"는, 동시에 가장 강한 "**디렉팅의 물증**"이다. LLM을 게임 루프에 넣어 게임 상태를 바꿀 권한은 **타입 수준에서** 못 넘게 막은 설계(§2) — 그게 "AI를 어디까지 믿고 어디서 끊는가"라는 디렉터의 일 그 자체이기 때문이다.

1. 한눈에 — 무엇에, 어떤 AI를, 어떤 권한으로

층위	도구	역할	초안 시점 상태
개발 지휘	Claude Code (Claude Opus)	설계 브레인스토밍, TDD 구현, 리팩터, 문서·리서치 오케스트레이션	전 기간 사용 (커밋·PR 이력에 그대로)
리서치 검증	다중 에이전트 교차·적대 검증 워크플로우	게임 전제(의료 통계·경제·소송)를 실제 근거로 팩트체크, 방어 가능한 부호만 채택	PR #10·#11·#18에 반영
문서 정합(lint)	다중 에이전트 스윕 + 적대 검증 워크플로우	문서가 코드·리서치의 현재 상태에서 얼마나 벌어졌는지 계측·교정	PR #36 (38건 중 16건 기각) · §3-4
게임 런타임 — 대사	Anthropic API	받는 콜 담당자의 대사만 생성(판정은 코드)	⏳ 극소판 예정 · 현재=결정론 폴백
게임 런타임 — 결말 해설	(현재) 결정론 순수 함수 / (예정) Anthropic API	결말 팩트 병치(장부·신문)	현재=결정론 팩트 병치 · LLM 해설=발전 가능성

권한의 원칙 한 줄: **게임 상태를 바꾸는 판정은 언제나 결정론 코드가 한다. AI는 그 결과를 연기하거나, 그 결과를 사람에게 설명할 뿐이다.**

2. 디렉팅 물증 ① — AI의 권한 경계를 설계하다 (판정 = 코드 / 대사 = LLM)

문제 — AI에 게임을 맡기면 무너지는 지점

이 게임의 메시지는 "당신이 아무리 애써도 구조의 벽은 안 뚫린다"이다. 플레이어는 응급 콜을 받는 **벽**이다. 그런데 수용/거절을 LLM에 그냥 맡기면, 콜 발신자가 절박하게 매달릴수록 LLM이 동조(sycophancy)해 "...알겠습니다, 받을게요"로 굴복한다. 그 순간 **게임의 메시지가 정반대로 뒤집힌다** — "사정이 딱하면 배후진료 없이도 받아진다"는 정답 퍼즐이 되어버린다. 이 대회에서 유일하게 치명적인 실패는 "AI가 게임의 논지를 갉아먹는" 이 지점이다.

설계 — 2콜 분리, 그리고 권한을 타입으로 못박기

수용/거절이라는 **게임 상태 전이**는 **절대 LLM이 정하지 않는다**. 병원의 숨은 제약(가용 병상·응급실 당직·배후진료 역량)을 읽는 **결정론 코드가 결과를 먼저 확정**하고, LLM은 "이미 정해진 결과"를 인물 대사로 **연기만** 한다.

이 원칙은 문서상의 다짐이 아니라 **함수 시그니처로 강제**돼 있다. 판정 함수에는 플레이어의 설득 텍스트를 받는 파라미터가 **아예 존재하지 않는다**.

```
// src/game/adjudicate.ts - 전원 수용/거절의 유일한 권한(2콜의 "판정콜")
// 입력은 병원의 숨은 제약과 환자뿐. 플레이어의 설득 텍스트는 이 함수의 파라미터로 존재하지 않는다.
export function adjudicateTransfer(hospital: Hospital, patient: Patient): TransferVerdict {
  if (hospital.beds <= 0) return { accepted: false, reason: 'NO_BED' }
  if (!hospital.hasErOnCall) return { accepted: false, reason: 'NO_ER_ONCALL' }
}
if (hospital.overcrowded) return { accepted: false, reason: 'ER_OVERCROWDED' }
if (!hospital.backupCare.includes(patient.requiredSpecialty))
  return { accepted: false, reason: 'NO_BACKUP_CARE' }
}
return { accepted: true }
}
```

설득 텍스트가 이 함수에 **타입으로 도달할 수 없으므로**, "매달리면 결과가 바뀐다"는 붕괴는 런타임에서 방어하는 게 아니라 **애초에 표현 불가능**하다. 병상 0이면 어떤 문장에도 `NO_BED` 가 유지된다는 것은 불변식 테스트로 잠겨 있다(\$3의 TDD).

대사는 이 확정된 판정을 받아 "연기"만 한다. 콜 발신자가 아무리 매달려도, 대사는 이미 정해진 결과(수용/하드락 사유)를 전할 뿐 판정을 바꾸지 못한다:

```
// src/game/dialogue.ts - 확정된 판정(수용/하드락 사유)을 "대사"로 옮기는 결정론 폴백
// 대사는 판정을 바꾸지 못한다. 이미 정해진 결과를 연기할 뿐이다.
// 입력은 콜과 판정 결과(disposition·reason)뿐 - 발신자의 애원 텍스트는 파라미터에 없다.
export function receivingLine(call: IncomingCall, disposition: CallDisposition, accepted: boolean, seed?: number, reason?: RejectionReason): string { ... }
```

라이브 LLM은 "설계한 빈칸"이 아니라 "채우고 통제된 슬롯"

- **현재:** 대사콜은 위 결정론 폴백이 돈다 → API 키 없이도 완주 가능.
- **극소판(예정):** 거절 대사 1종을 실제 Anthropic 왕복으로 렌더한다. 무키·타임아웃·호출상한 초과 시 **자동으로 위 폴백으로 강등**. 이로써 게임 루프 안에 살아있는 AI가 한 조각 들어가되, 그 AI는 여전히 판정 권한이 없다.
- **발전 가능성(x8):** 대사콜 풀버전 + LLM 결말 해설 + **정합성 테스트(코드 판정 == 최종 게임 상태)**. 사전 과제 필수가 아니라, 디렉팅 물증을 더 두껍게 만드는 상위 레이어다.

왜 이게 디렉팅인가 — LLM을 루프에 넣는 건 누구나 한다. 이 설계의 요점은 **넣은 뒤 어디서 끊었는가**다. 상태 전이 권한을 코드에 잠그고(타입 하드웨어), 실패 시 결정론으로 강등하는 경로를 항상 손에 쥐어 채 LLM을 전량 업사이드로만 엮었다. "AI를 어디까지 믿고 어디서 끊는가" — 그 경계선을 그은 사람이 디렉터다.

3. 디렉팅 물증 ② — AI를 지휘해 만든 개발 과정

3-1. 거버넌스 = 디렉팅 명세 (레포에 상주하는 "지휘 규칙")

이 저장소에는 AI를 지휘하는 규칙 시스템이 문서로 상주한다. 공개 레포에서 그대로 확인 가능하다.

- **AGENTS.md / CLAUDE.md** — 프로젝트 헌법. 예: "이 Next.js는 breaking change가 있으니 코드 작성 전 `node_modules/next/dist/docs/` 의 해당 가이드를 먼저 읽어라" — AI가 낯은 학습지식으로 코드를 쓰지 못하게 못박는 가드.
- **작업 추적 3종** — `plan.md` (앞으로 할 일) · `changeLog.md` (완료 기록) · `troubleshooting.md` (함정+재발방지). 매 작업(대체로 PR)마다 세 문서를 한 세트로 갱신해, "무엇을/왜/어떤 함정"이 이력으로 남는다. **AI의 작업을 감사 가능하게 만드는 장치.**
- **인코딩 함정 차단** — `.commit-msg-tmp` 를 gitignore(PowerShell 5.1 한글 커밋 깨짐 회피 절차의 잔재가 커밋되는 걸 원천 차단).

이 규칙들은 "AI 에이전트 설계서 / 디렉팅 명세서"(본선 제출물)의 사전 과제판이다 — AI에게 무엇을 시키고, 무엇을 금지하고, 어떻게 기록하게 할지를 미리 못박은 명세.

그 거버넌스를 실제로 감사했다 — 준수 100%, 집계 0회

규칙을 "감사 가능하게 만드는 장치"라고 부르려면 감사를 한 번은 돌려봐야 한다. 이 저장소엔 그 목적의 규칙이 하나 더 있다 — 커밋 트레일러로 사용 스킬을 자가보고하게 한 규칙(`CLAUDE.md`, PR #25 도입). **2026-07-17, 도입 15커밋 만에 처음 집계를 돌렸다.**

항목	실측
규칙 도입(69494bd) 이후 커밋	15
트레일러를 남긴 커밋 — 준수율	15 = 100%
<code>git log --grep</code> 으로 집계되는 커밋	15 = 100%
<code>git interpret-trailers --parse</code> 로 파싱되는 커밋	2 = 13%
집계를 실제로 돌린 횟수	0 — 오늘이 처음

(a) 빈도 — TDD 9회 · brainstorming 5회 · `none` (스킬 미사용) 6회 · 나머지 4종 각 1회.

(b) 미사용 — 설치 카탈로그와 대조하니 개발 프로세스 스킬 14종 중 9종을 한 번도 안 썼다. 그중 셋이 눈에 띈다: `using-git-worktrees` (워크트리 사고 T-038을 2회 내는 동안 미사용) · `systematic-debugging` (T-039~T-043 디버깅을 전부 맨손으로) · `verification-before-completion` (T-043 — `tsc` 0건·테스트 444 green인데 버튼이 아무 일도 안 한 무성 버그).

(c) 형식은 100% 지켜졌는데 파싱 경로는 13커밋 동안 죽어 있었다. `Skill-benefit` 과 `Co-Authored-By` 사이에 빈 줄을 하나 넣었더니 git이 마지막 문단만 트레일러로 보고 앞을 통째로 버렸다. 규칙 위반이 아니다 — 규칙엔 빈 줄 이야기가 없었고 예시는 처음부터 옳았다. 준수되면서 목적만 증발했고, 읽는 사람이 없으니 아무도 몰랐다.

결론을 과장하지 않는다. " using-git-worktrees 를 안 써서 T-038이 났다"는 인과는 **틀렸다**. 그 스킬을 열어 보니 base 최신화를 한 마디도 하지 않는다(git worktree add "\$path" -b "\$BRANCH" — base 인자 없이 현재 HEAD에서 떴다). **썼어도 안 막혔다**. 미사용은 상관이지 원인이 아니다 — \$5의 '감수' 원칙(없는 걸 있다고 부르지 않기)이 여기에도 적용된다.

그래서 무엇을 했나 — 규칙을 더 짧게 쓰지 않고 사다리를 한 칸 올렸다. T-038은 승격 3단계 중 두 칸을 이미 썼다: 프로젝트 troubleshooting.md → 글로벌 규칙("워크트리 분리 전 로컬 main 최신화"). **그런데도 2회 재발했다.** 소프트웨어 규칙은 통하지 않는다가 이 감사의 결론이므로 남은 칸은 하나다 — **혹 하드가드**. block-stale-base.ps1 (PreToolUse)이 "작업 브랜치 + 자체 커밋 0 + origin보다 뒤쳐짐"이면 편집을 **거부**한다. 커밋을 하나라도 하면 다시 걸리지 않는다 — **되돌릴 수 있는 시점에만 말한다**. (개인 환경 전역 혹이라 이 저장소 밖에 있다 — 위 다른 물증과 달리 공개 확인은 안 된다.)

왜 이게 디렉팅인가 — 규칙을 쓰는 건 누구나 한다. 요점은 자기 규칙을 감사해서 그게 안 듣는다는 데이터를 만들고, 그 데이터에 따라 규칙을 버리고 하드가드로 옮긴 것이다. 이 감사가 낳은 규칙 하나: **형식 규약에는 검사기를 같이 만든다**. 검사기 없는 규약은 죽는 게 아니라 썩는데, 썩음은 준수율로 안 잡힌다.

3-2. TDD Red → Green — 판정·타이머·하드락을 LLM 없이 먼저 잠근다

시뮬레이션 코어는 전부 **테스트 우선**으로 지었다(실패를 먼저 확인 → 구현 → 통과). 현재 **시물 코어 233개 테스트(9파일)** 그린, tsc --noEmit · next build 통과.

코어	파일	무엇을 잠갔나
수용 판정	adjudicate.test.ts	4종 거절 사유 · 병상 0·배후진료 하드락 불변식 (어떤 입력에도 안 뚫림)
받는 흐름·경제	receiving.test.ts	응급 4종(분만·뇌출혈·중증외상·심근경색)이 배후과 없으면 하드락 · 콜 경제 부호(급여·원가미달)
받는 대사	dialogue.test.ts	판정→대사만, 대사가 판정을 못 바꿈 · 과별 하드락 사유 정확(과 뒤바뀌면 거짓말)
세션 상태기계	session.test.ts	7일 진료→주간 결산 직행 전이, 받은/돌려보낸 응급 집계
아침 신문	news.test.ts	못 받은 응급의 종류별 헤드라인 + 실제 사건 실명 토큰 가드
병원 장부	ledger.test.ts	흑자↔필수와 채용 0의 병치, 파생값 검증

핵심은 순서다 — **게임의 논지를 지탱하는 불변식(설득으로 벽이 안 뚫린다)을 LLM을 붙이기 전에 코드로 먼저 증명**했다. LLM은 이 잠긴 코어 위에서만 논다.

3-3. 리서치 → 도메인 반영 — AI가 낸 안을 사람이 검증·기각한 로그

AI를 아이디어 뱌는 기계로 쓰지 않고, **AI가 낸 것을 다른 AI들과 사람이 반증**하게 했다.

- **PR #10** — 다중 에이전트 리서치 + 적대적 검증으로 게임 전제(STEMI·응급실 뽕뽕이)를 실제 통계로 팩트체크 → 방어 가능한 것만 팩트시트로.

- **PR #11** — 그 리서치가 도메인 모델 자체를 갈아엎었다: "지배 병목은 병상 없음이 아니라 배후진료 불가"라는 근거로, 뭉뚱그린 `onCallSpecialties` 를 `hasErOnCall` (초기 수용) + `backupCare` (최종치료)로 분리하고 거절 사유를 4종으로 재설계.
- **PR #18** — 필수과 소송·방어진료 리스크의 "부호"를 5갈래·적대 검증으로 확인(검증 불가 수치는 배제, 방향만 채택).

이 세 PR이 보여주는 건 "AI가 만들어줬다"가 아니라 **"AI 산출물을 어디서 신뢰하고 어디서 기각했는지"의 판단 로그다**. 디렉터의 일은 생성이 아니라 이 취사선택이다.

3-4. 남의 패턴을 채택하지 않기로 한 판단 — LLM Wiki 대조 (PR #36)

우리는 LLM Wiki를 구현하지 않았다. 대조해보고 왜 이 도메인엔 안 맞는지 알아냈다.

Andrej Karpathy가 2026-04에 제안한 LLM Wiki는 "지식을 RAG처럼 매 질문마다 재발견하지 말고, LLM이 마크다운 위키를 소유·유지해 한 번 컴파일하라"는 패턴이다(3층: raw sources / wiki / schema, 3 op: ingest / query / lint). 이 저장소는 겉보기에 그것과 닮았다 — 상호링크된 마크다운 문서군, LLM이 쓰고 사람은 읽는 소유 구조, 세션 시작 혹은 frontmatter에서 재생성하는 자동 인덱스, 함정을 흑으로 승격시키는 사다리(§3-1). **그래서 대조해봤다.**

결과: 아니다. 46에이전트 워크플로우(조사 5 → 매핑 → 3렌즈 적대검증 → 구멍분석)로 9개 요소를 대조한 결과 — **strong 0 / partial 1 / stretch 6 / 명시적 부재 2**. 검증 전 강도가 한 칸도 유지되지 못하고 전부 하향됐다.

Karpathy 요소	우리 것	적대검증 후
schema	글로벌 <code>CLAUDE.md</code> (259행)	partial — 형식·소유권은 부합하나 규율 대상이 위키가 아니라 git-TDD
wiki / log.md / index.md / ingest / raw-sources / cli-tools	research 4종 · changeLog · README 표 · 리서치 워크플로우 등	stretch ×6
query · lint	—	없음

원인은 게으름이 아니라 **트리거의 방향이다**. Karpathy 패턴의 전제는 소스가 도착하는 것(**source-push**)이다 — 기사를 떨구면 LLM이 읽고 위키 10~15장을 갱신한다. 우리 트리거는 반대다: 코드가 근거를 필요로 할 때 리서치를 발주한다(**feature-pull**). 그래서 기존 페이지를 재검토할 계기가 **원리적으로 생기지 않고**, 지식이 문서가 아니라 **코드로 굳는다**.

증거 넷 — 전부 이 저장소의 공개 이력에서 확인 가능하다:

1. `claude-docs/learning-notes.md`의 **생애**. 글로벌 규칙이 "필수 셋업 1번"으로 못박은 유일한 지식 축적 페이지가 부트스트랩에 빈 골격으로 태어나 **22커밋 동안 한 줄도 못 쌓고**(`93f18ea` → `aba6d77`, 내용 변경 0건) PR #22에서 "이 프로젝트 미사용"으로 삭제됐다. **규칙이 없어서 안 쌓인 게 아니라, 규칙이 있는데도 안 쌓였다.**
2. **PR #18** — 29에이전트를 돌리고 총 2파일 커밋. 기존 리서치 3종 갱신 **0건**.
3. **PR #36이 고친 것 자체** — 리서치의 지배 병목 교정이 **코드로는 흘러갔지만**(`adjudicate.ts`의 `NO_BACKUP_CARE`, PR #11) **문서로는 안 갔다**. `game-concept.md`는 부트스트랩 이후 **35커밋 무갱신**이라 게임 1막(설립 위저드·하루 자리 제한·7일 달력) 전체를 모르고 있었다.

4. 그리고 진단이 예측한 실패가, 진단하던 PR에서 그대로 일어났다. PR #36의 lint는 **stale한 base**(#33 시점)에서 돌아, 그 사이 머지된 #34-#35를 모른 채 120에이전트가 낡은 코드를 읽었다 → **낡은 문서를 고치면서 #35가 이미 폐기한 개념("분기 손익")을 새로 써넣었다**(13건 중 3건 오염, 사후 발견·재작성 → T-038). 적대 검증 3렌즈도 못 막았다 — 검증은 "트리 안에서의 정합성"만 봤지 "트리 자체가 최신인가"는 아무도 보지 않았다.

그래서 무엇을 했나 — **채택이 아니라 계측**. 패턴을 이식하는 대신 그 패턴이 짚어준 결함 하나를 실제로 고쳤다 (PR #36, 120에이전트 lint → **38건 중 16건 기각**). 반대로 **채우지 않기로 한 것도 명시했다** — docs/raw/ 신설(저작권·비용 대비 무의미), ingest 규칙 성문화(소스가 안 오는데 규칙만 넣으면 learning-notes 실패의 재연), 위키 검색 도구(Karpathy 본인이 optional, 우리 규모엔 ripgrep 으로 충분). **도구를 위한 도구를 만들지 않았다**.

왜 이게 디렉팅인가 — 유행하는 AI 패턴을 가져다 붙이는 건 누구나 한다. 요점은 **붙이지 않기로 한 근거를 데이터로 만든 것이다**. "우리도 LLM Wiki를 한다"고 쓰는 편이 유리해 보이지만 그 주장은 PR #18의 2파일 커밋 하나로 뚫린다. 그래서 위 표의 빈 칸(query-lint)은 stretch로 채우지 않고 **빈 칸으로 뒀다** — §5의 '감수' 원칙(없는 걸 있다고 부르지 않기)이 여기에도 그대로 적용된다. 남의 패턴에 부분 점수를 구걸하는 대신 우리가 실제로 한 것에 정확한 이름을 붙였다: **이건 LLM Wiki가 아니라 "LLM이 운영하는 개발 워크플로우"다**. 무엇을 쓸지 정하는 것만큼 무엇을 안 쓸지 정하는 것도 디렉터의 일이고, 후자는 대개 기록에 남지 않는다. 그래서 남겼다.

4. 게임 속 AI ③ — 결말: 설명하지 않는 병치 (현재 결정론 / LLM 해설 = 발전 가능성)

결말은 "무엇이 죽었나"를 **설명하지 않는다**. 그 주의 팩트만 라벨+숫자로 병치하고, 플레이어가 스스로 잇게 한다 ("보여주지 말고 겪게"). 결말 화면은 두 팩트 면을 나란히 둔다:

- **누적 병원 장부(구현됨)** — ledger.ts : 미용·검진으로 흑자인데 그 환자에게 필요했던 필수과(순환기·산부인과 등) 채용은 0. 이 0은 하드코딩이 아니라 backupCare (=하드락 사유 NO_BACKUP_CARE 의 뿌리)에서 **파생**된다 → "왜 배후진료 불가였나"의 경제적 뿌리를 조작 없이 드러냄.
- **그 주 신문(구현됨)** — news.ts : 못 받은 응급들이 종류별(심근경색·뇌졸중·중증외상·분만) 헤드라인으로 실린다(해석 카피 0). 실제 사건과 겹치지 않도록 설명 토큰 가드가 잠겨 있다(§5 톤 가드레일).
- **발전 가능성(x8)** — LLM이 플레이 로그를 근거로 "이 결과를 만든 구조 변수 + 반사실"을 해설. 구조화 출력 스키마와 톤 가드레일(특정 집단 비난 금지) 설계는 잡혀 있다.

⌚ [자리표시자 — x8 후: 실제 LLM 결말 해설 프롬프트·구조화 출력·반사실 화면 캡처]

5. 안전 · 정확성 · 비용

- **의학적 정확성** — 게임 내 수치는 각색이되 **부호·대소(적자↔흑자, 지배 병목의 방향)**는 근거를 지킨다. '감수'라 칭하지 않고 **출처 각주 + 게임적 각색 고지**로 처리(검증 불가 수치는 출처 명시 범위로만). 근거: essential-care-economics · essential-care-litigation-risk · medical-system-grounding · stemi-factsheet.

- **톤 가이드라인** — "특정 직군·집단을 비난하지 않고 **구조적 공백만 근거**"를 결정론 템플릿과 (예정) LLM 프롬프트 양쪽에 고정.
- **키·비용 안전** — LLM 키는 **서버 전용**(`NEXT_PUBLIC_` 금지 → 브라우저 번들 노출 차단, 서버 API 라우트에서만 사용). 판당 호출 상한(6~10)을 코드에 못박고, 무키·타임아웃·상한 초과 시 결정론 폴백으로 **자동 강등**.
- 🕒 [자리표시자 — 유료 API 비용 부담 주체는 대회 FAQ 확인 후 확정(문의: info@nhn-nan.com). 미설정 시 극 소판/ x8 은 스킵 전제, 본 문서는 설계+로컬 시연 중심으로 성립.]

6. 실제 프롬프트 — 개발 과정의 지시 로그

여기 실린 것은 **개발 세션에 실제로 입력된 원문**이다(2026-07-15 ~ 07-22, 세션 로그 전수 파싱). 오타·구어체를 고치지 않고 그대로 싣는다 — 다듬으면 그 순간 "사후에 쓴 문서"가 되기 때문이다.

6-0. 전수 통계 — 무엇을 지시하고 무엇을 위임했나

7일간 프로젝트·워크트리 8개 세션에 입력된 사용자 프롬프트를 전부 추출했다(도구 알림·시스템 주입 제외).

항목	실측
총 프롬프트	257개 (7일, 총 18,347자)
길이 중앙값 / 평균	21자 / 71.4자
20자 이하 — "머지하자"."진행해"급 승인	128개 (49.8%)
100자 이상 — 방향을 바꾼 지시	45개 (17.5%)

절반이 20자 이하다. 이건 게으름이 아니라 위임 구조의 지표다 — 규칙(§3-1)과 잠긴 코어(§3-2)가 서 있으면 대부분의 톤은 승인으로 끝나고, 사람의 언어는 **방향이 갈리는 45개 지점**에 몰린다. 아래 6-1~6-3은 그 45개에서 골랐다.

6-1. 디렉팅 프롬프트 5선 — 지휘가 실제로 일어난 자리

① 심사 기준을 다시 읽고 문서의 축을 뒤집은 지시 (7/16)

"사전 과제 제출물에서 AI 활용 기술 문서에 관한 내용이 게임에 AI를 어떻게 녹였냐가 아니라 AI를 활용해서 어떻게 이 게임을 만들었냐를 보는거 같거든? 지금 넌 이 평가 기준을 이미 알고있어? 뭔가 내가 경험한 느낌은 AI를 어떻게 게임 속에서 하나의 기능으로 활용할지에 무게를 두는거 같아서,"

→ 이 문서의 §0(디렉팅=주 / 게임 속 AI=종)이 여기서 나왔다. 그 전까지 AI는 "게임에 넣을 기능"으로 다뤄지고 있었다. 사람이 요강을 다시 읽고 축을 바꿨다.

② AI 해커톤에서 '눈에 보이는 AI'를 스스로 삭제한 결정 (7/18)

"1. 뽕뽕이 경험과 대화형은 사실 아무 상관이 없다고 생각해. 왜냐하면 사실 다른 병원에 환자 받아달라고 전화할 때 어떻게 말을 하냐가 중요하지 않잖아. 서비스업도 아니고. 그냥 기존 시스템을 일단 유지하되 거기서 사용자가 대사를 넣을 수 있던 박스만 삭제해. 2. 라이브로는 뒷단에서 AI가 돌아가는걸 보여주지 힘들겠지. 이걸 문서로 강조하고 라이브 동영상의 AI가 노출되지 않는다는 단점은 감안하자."

→ 자유 텍스트 설득 입력은 **영상에서 AI가 가장 잘 보이는 자리**였다. 그걸 "현실의 전원 거절은 화술로 뒤집히지 않는다"는 이유로 지웠고, 심사에서 불리할 수 있음을 알면서 감수했다. §2의 권한 경계가 문서상의 다짐이 아니라는 증거가 이 프롬프트다. 이를 전 같은 방향의 선행 결정도 있었다 — "*대화 LLM은 사실 없어도 될거같은 하거든. [...] plan에 넣어서 발전 가능성으로만 기록해두고 일단은 우선순위에서 많이 뒤로 미뤄도 될거같아.*"(7/16)

③ AI 산출물의 근거를 의심한 2연타 (7/16)

"혹시 지금 게임을 만드는데. 현재 대한민국 의료 시스템에 대한 문제점을 먼저 파악하고 게임의 방향성을 잡은거야? 아니면 그냥 막연히 진행하고 있는거야??"

→ "실제로 리서치 하고 구멍 매우자. 이 게임은 의도가 중요한 게임인데 리서치 없이 진행하면 나중에 의도에 타격이 있을 가능성이 있을거같아."

→ 첫 문장이 **AI가 그럴듯하게 만든 것과 근거가 있는 것을 구분하**라는 요구다. 이 두 턴이 §3-3의 리서치 축(PR #10-#11-#18)을 열었고, 그 결과 도메인 모델의 지배 병목이 **병상 없음** → **배후진료 불가**로 교체됐다.

④ AI의 자기 규칙 준수를 감사한 질문 (7/20)

"근데 요즘들어 트러블슈팅 작성이 거의 없는데 트러블이 없어서 작성을 안한거야 아니면 트러블슈팅 작성을 너가 생각을 안하고 있는거야? 왜냐하면 저번에 넌 트러블슈팅을 작성 안하고 메모리를 우선적을 작성했던적이 있거든."

→ "안 일어난 것"과 "AI가 못 알아챈 것"을 구분해서 물었다. **후자였다** — 실측 결과 T-054 이후 8 PR 동안 트러블슈팅 0건. 서버에이전트가 디버깅을 흡수하면서 진행 ledger가 **완료**만 추적하고 **만난 함정**을 안 남긴 구조적 공백이었고, 이 질문이 T-055 등재와 CLAUDE.md의 「SDD 종료 trap 스위치」 규칙을 낳았다. §3-1의 감사(준수 100%.집계 0회)와 같은 계열의 개입이다.

⑤ AI 사용 자체를 계속 대상으로 만든 규칙 신설 (7/16)

"Claude.md에 규칙을 하나 추가하자. 이 프로젝트에만 적용되는 규칙인데 쓸모있다 싶으면 승격할거야. 작업 중에 사용된 스킬, plugins을 깃 커밋 메시지에 명시해줘. 그렇게해서 해당 스킬이 얼마나 자주 사용되는지, 또는 있는데 사용되지 않는 스킬은 뭔지. 그 스킬을 통해서 어떤 이점을 봤는지를 파악하는데 도움을 얻기 위함이야. 기술적으로 가능해?"

→ §3-1이 감사한 **그 규칙**의 발주 프롬프트다. "쓸모있다 싶으면 승격할거야"라는 조건부 도입이 곧 프로젝트 → 글로벌 → 혹 하드가드 사다리의 첫 칸이었다.

6-2. 같은 벽을 네 번 세우다 — show-don't-tell

AI는 친절하게 설명하려는 기본 성향이 있고, 그 성향은 이 게임의 논지("겪게 하고 설명하지 않는다")를 조용히 갉아먹는다. 한 번의 지시로는 안 잡혀서 **네 번 같은 벽을 세웠다**.

날짜	원문(발췌)	무엇을 막았나
7/16	"지금 게임을 보면 구조를 바꿨다면 더 좋았을 거라는걸 직접적으로 보여주고 있거든. 그냥 글로 보여주잖아. 이거 없애자. [...] 이렇게 내놓고 구조적입니다 라고 글로 보여주고 싶지 않아. 이건 좋은 게임이 아니야. "	결말의 해석 카피 → §4 "해석 카피 0"의 뿌리
7/17	"수익 예상이 높은지 낮은지, 소송 리스크가 큰지 적은지는 명시적으로 보여주지마. 플레이어가 경험을 통해서 알도록 유도하게."	개원 위저드의 정답 표시(고르기 전에 답을 알려주는 UI)
7/19	"필수의로 단어 설명은 좋은데, 추가적으로 그 단어와 현재 의료시스템의 문제까지 엮어서 설명하는건 빼. 단어설명만 사용자에게 전달하라는거야."	용어 툴팁에 논평이 없히는 것
7/20	"병원을 키울 수 있도록 장치를 추가하자. 단, 대한민국 의료 시스템의 딜레마를 생각하고 시스템을 만들어야 해. 병원 규모가 커진다고 해서 의료시스템에 문제가 사라지지 않는거잖아. "	성장 재미가 메시지를 해소해버리는 설계 → 제로섬 재투자 루프

마지막 것은 방어가 아니라 **발주 조건**이다. "성장을 넣되 성장으로 문제가 풀리면 안 된다"를 기능 요청과 같은 문장에 넣었고, 그래서 전국 유한 인력 풀(돈이 있어도 못 뽑음)이 나왔다.

6-3. 오케스트레이션 발주 — 24자가 3세션 삼각검증이 되기까지

"후보 구현 우선순위 뽑기 작업을 프롬프트로 줘 / 세 세션에서 진행하게" (7/18, 24자)

이 짧은 지시의 요구는 두 겹이다. (1) 프롬프트를 **AI가 쓰게** 하고, (2) 그 프롬프트를 **서로 모르는 3개 세션에 독립으로** 돌려 교차검증한다. 산출된 지시서(1,933자)는 채점축 4개(메시지 기여·메커닉 적합·구현비용·팩트/톤 안전)와 가중 공식 $M \times 2 + F + C + S$, 그리고 "다른 세션 결과를 절대 참고하지 말 것 / 출력 형식을 바꾸지 말 것"(병합 가능성 확보)을 못박았다.

⚠ **정확한 귀속** — 그 1,933자 지시서는 **사람이 아니라 AI가 작성했다**. 사람이 한 것은 "프롬프트로 줘 / 세 세션에서" 24자와, 세 결과를 받아 병합·판정한 것이다. 없는 걸 있다고 부르지 않는다(§5) — 이 파이프라인의 이름은 "**사람이 검증 구조를 발주하고, AI가 지시서를 쓰고, 사람이 결과를 병합한다**"이지 "사람이 정교한 프롬프트를 썼다"가 아니다.

같은 계열의 판단 프롬프트 둘을 덧붙인다.

- **도입하려던 AI 패턴을 스스로 철회** (7/17) — "LLM Wiki를 구축하기 위해서 이런저런 작업을 했는데, 내가 원하는게 LLM Wiki가 아니었던거 같아. [...] 이 아이디어에 허점 파악해봐. 일단 당장 내 머리에 떠오른 허점은 링크를 AI한테 맡기는데, 여기서 잘못된 판단으로 인해 연결 되어야 하는게 연결되지 않는 문제가 있을 수 있다고 생각해." → 하루 전에는 "NHM 해커톤 평가사항인 AI 활용도 측면에 어필하고 싶은게 생겼어. LLM Wiki를 활용했다는걸 넣고 싶거든." (7/17)이었다. **어필용으로 넣으려던 패턴을 직접 철회하고, 자기 대안의 허점까지 AI에게 먼저 반증시켰다** → §3-4.
- **근거가 흐리면 구현을 멈춤** (7/20) — "현실에서 내과 진료 과목 상대로 소송이나 언론에 논란으로 다뤄진적이 있어??" → (팩트체크 결과 회색지대) → "내과는 애매한 부분이 많네. 피부과처럼 안전한 느낌을 내과에 줄 생각은 전혀 없거든. 일단은 법적 리스크를 구현하지는 말되, 나중에 좀 더 리서치하고 구현하게 문서로 남겨는"

ㄴㄹ." → 근거의 부호가 안 잡히면 **기능을 넣지 않고 보류로 남긴다**. §5의 원칙이 실제 기능 결정에서 작동한 사례.

6-4. 아직 없는 것

 [자리표시자 — 극소판/ x8 착수 후: (a) 전원 대사콜 시스템/유저 프롬프트 전문 + 실제 응답 23개, (b) 무카 폴백으로 강등되는 순간의 로그 1개를 첨부해 "강등이 실제로 작동함"을 증거화.] 위 6-16-3은 **개발 과정 (B축)의 지시 로그**이며, **게임 런타임(A축)의 프롬프트↔응답 로그는 실LLM 착수 전이라 아직 없다**.

7. 외부 에셋 · 오픈소스 출처

구분	항목	비고
프레임워크·툴	Next.js 16 (App Router), React, TypeScript, Tailwind CSS, vitest	각 오픈소스 라이선스 준수
AI	Anthropic Claude — 개발(Claude Code) + 런타임 API(대사·결말 해설)	
에셋	현재 UI는 커스텀 텍스트·SVG 중심	 [자리표시자 — 외부 이미지·폰트·사운드 사용 시 출처·라이선스 여기 명시. 유의사항 필수 항목]

관련 문서

 **시각화·PDF판 — 디렉터의 운영체제** (라이브 아티팩트 · 최종 브라우저 인쇄로 PDF)

[game-concept](#) · [submission-plan](#) · [plan](#) · [changeLog](#) · [troubleshooting](#)

문서 상태: 작업 초안 (2026-07-22 갱신 — §6 채움). 런타임 LLM 로그·스크린샷·최종 수치는 P7(8/3~8/7)에 채워 확정.